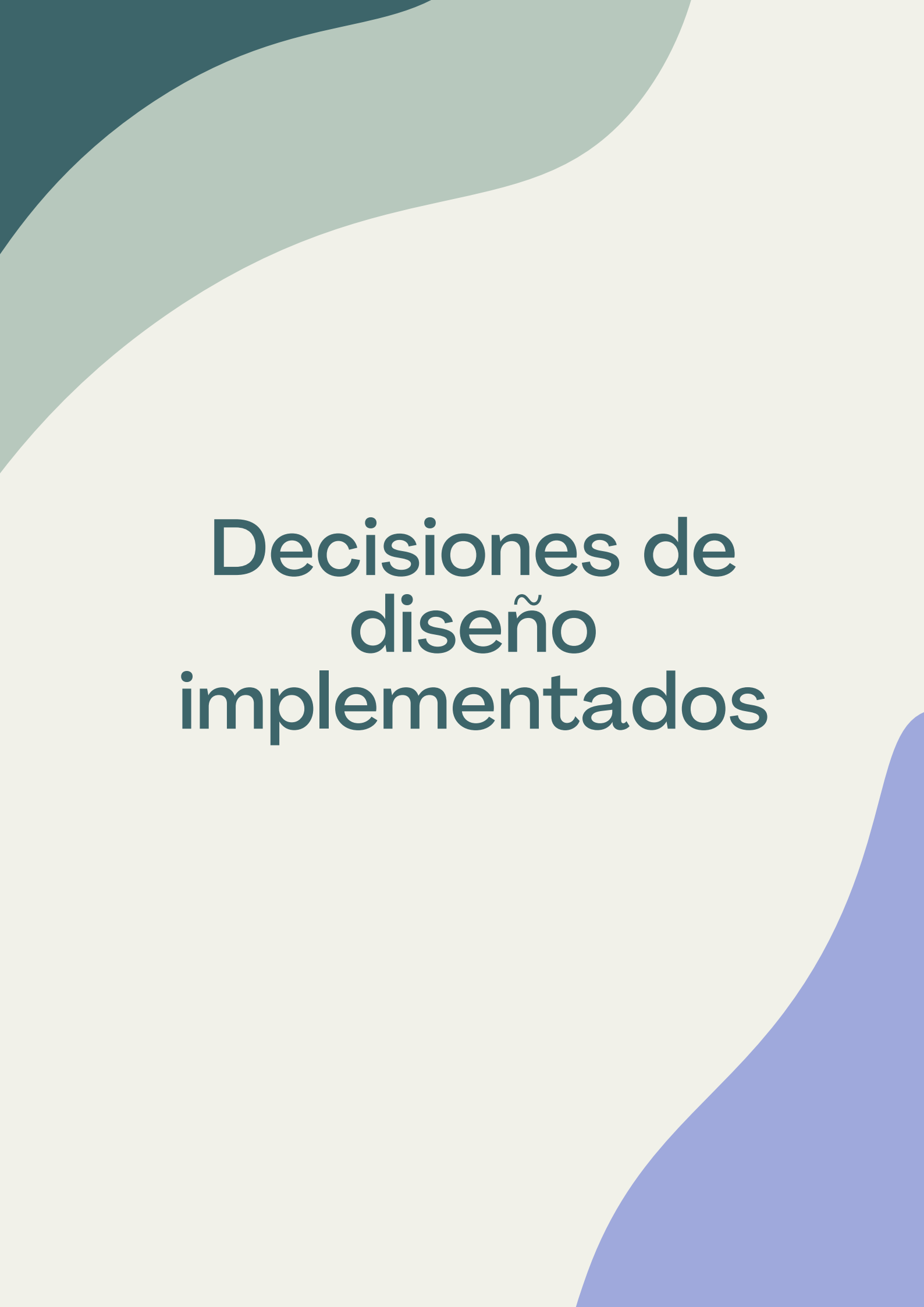


Trabajo Integrador Objetos II

ARENA CAMILA: ARENABELU@GMAIL.COM

MORETTI LUCA: LUCAMORETTIAR2005@GMAIL.COM

CARBONE MARIANELA: MARIANELACARBONE3@GMAIL.COM



Decisiones de diseño implementados

Las decisiones de diseño se tomaron en conjunto durante la elaboración del UML, eligiendo la mejor opción de qué patrón aplicar en cada caso según el libro de Gamma.

Estas decisiones permitieron lograr un diseño coherente, mantenible y alineado con las buenas prácticas que se mostraron en la materia.



Patrones de diseño implementados

Template Method

Utilizamos el template method para servicios porque todos implementan la misma estructura general pero cada uno calcula el costo de diferente manera

Clase abstracta: servicio

Clases concretas:

Lavado,Electricidad,Pesado,Almacenamiento

Composite

Se decidio usar el patron Composite para modelar los filtros de busqueda ya que no modifica la lógica de búsqueda y trata filtros simples y combinados de la misma manera.

Componente:BuscadorDeTrayectosStrategy

Hoja:BuscadorPorPuertoDestino
BuscadorPorFechaDeLlegada
BuscadorPorFechaDeSalida

Compuesto:BuscadorAND BuscadorOR

Cliente:TerminalGestionada

Strategy

En este caso tambien se considero que era un Strategy porque eran distintas formas de resolver un mismo problema ademas de que el modelo queda abierto para agregar nuevos filtros a futuro

Estrategia: BuscadorDeTrayectosStrategy

**Estrategia Concreta: BuscadorPorPueroDestino
BuscadorPorFechaSalido
BuscadorPorFechaLlegada
BuscadorOR
BuscadorAND**

Context: TerminalGestionada

Strategy

Utilizamos el patron strategy para el ejercicio de devolver el mejor circuito porque va a haber diferentes formas de resolver el mismo problema y ademas puede quedar abierto para agregar nuevos modelos

**Estrategia: MenorTiempoTotal MenorPrecioTotal
MenorCantidadDeTerminales**

Estrategia Concreta: MejorCircuitoStrategy

Contexto: TerminalGestionada

Observer

Se decidio aplicar el patron
Observer para notificar a los
actores portuarios cuando
reciben la carga

Observer: ActorPortuario

Observadores: Shipper Consignee

Sujeto: Sujeto

Sujeto Concreto: TerminalGestionada

State

Se decidio usar el patron
Stateya que el
comportamiento de Buque
cambia segun su estado
interno y cada estado define
su comportamiento

Contexto:Buque

Estado:EstadosDelBuque

**Subclases de estado: Outbout Inbound
Arrived Working Departing**

Visitor

El metodo visitor es el adecuado ya que nos permite generar distintos tipos de reportes sobre el mismo objeto

Visitor:ReporteVisitor

Element:ReporteVisitable

**ConcreteVisitor:ReporteAduanaVisitor
ReporteMuelleVisitor
ReportBuquerVisitor**